Streamlining Ticket Assignment for Efficient Support Operations.

Project Documentation format

1. Introduction

• **Project Title:** Streamlining Ticket Assignment for Efficient Support Operations.
• **Team Members:** Lakshmanan S, Gokulakannan S, Jegan R, Aravind A, Bavithran M.

---

2. Project Overview

• **Purpose:** Briefly describe the purpose and goals of the project.
This project aims to automate and optimize the process of assigning support tickets to agents. It helps improve efficiency, reduce response time, and ensure balanced workload distribution among support staff.

• **Features:**

- User authentication (Admin & Agents)
- Ticket creation and management
- Automatic ticket assignment
- Priority-based ticket handling
- Ticket status tracking (Open, In Progress, Resolved)
- Dashboard for monitoring tickets
- Search and filter options

---

3. Architecture

• **Frontend:** Describe the frontend architecture using React.
The frontend is built using React with a component-based architecture. It uses functional components and hooks for state management. React Router is used for navigation, and Axios is used to communicate with backend APIs.

• **Backend:** Outline the backend architecture using Node.js and Express.js.
The backend is developed using Node.js and Express.js to create RESTful APIs. Middleware is used for authentication and request handling. Business logic for ticket assignment is implemented on the server side.

• **Database:** Detail the database schema and interactions with MongoDB.
MongoDB is used as the database with Mongoose for schema modeling. Collections include Users and Tickets. Tickets are linked to users (agents) for assignment and tracking.

---

4. Setup Instructions

• **Prerequisites:**

- Node.js
- MongoDB (local or Atlas)
- npm or yarn

• **Installation:**

Step-by-step guide to clone, install dependencies, and set up the environment variables.

```
git clone https://github.com/your-repo/ticket-system.git
cd ticket-system
```

Install dependencies:

```
cd client
npm install
cd ../server
npm install
```

Create `.env` file in server folder:

```
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_secret_key
PORT=5000
```

---

5. Folder Structure

• **Client:** Describe the structure of the React frontend.

```
client/
├── src/
│   ├── components/
│   ├── pages/
│   ├── services/
│   ├── App.js
│   └── index.js
```

• **Server:** Explain the organization of the Node.js backend.

```
server/
├── controllers/
```

```
│── models/
│── routes/
│── middleware/
│── config/
│── server.js
```

---

6. Running the Application

• Provide commands to start the frontend and backend servers locally.

o **Frontend:** npm start in the client directory

o **Backend:** npm start in the server directory

---

7. API Documentation

• Document all endpoints exposed by the backend.
• Include request methods, parameters, and example responses.

Example:

- **POST /api/tickets** – Create a ticket
- **GET /api/tickets** – Get all tickets
- **PUT /api/tickets/:id** – Update ticket status
- **POST /api/users/login** – User login

---

8. Authentication

• Explain how authentication and authorization are handled in the project.
Authentication is implemented using JWT (JSON Web Tokens). Users log in and receive a token, which is used to access protected routes.

• Include details about tokens, sessions, or any other methods used.
Passwords are hashed using bcrypt, and role-based access (Admin/Agent) is enforced using middleware.

---

9. User Interface

• Provide screenshots or GIFs showcasing different UI features.
The UI includes login page, dashboard, ticket creation page, and ticket tracking interface.

---

10. Testing

• Describe the testing strategy and tools used.
Testing is performed using Postman for API validation and manual testing for frontend functionality.

---

11. Screenshots or Demo

• Provide screenshots or a link to a demo to showcase the application.

---

12. Known Issues

• Auto-assignment may not always perfectly balance workload
• Minor UI issues on smaller screens

---

13. Future Enhancements

• AI-based ticket assignment
• Real-time notifications
• Email integration
• Advanced analytics dashboard